Personal, Relevant Background, and Future Goals Statement

In Fall 2024, I left the SHPE National Convention more afraid for my future than I had ever been. Thousands of Hispanic engineers and computer scientists were there, many just as passionate and hardworking as I was, and I realized that passion was not going to set me apart. I had almost nothing to show a company beyond my coursework. Hiring me would have been a blind gamble. The students who stood out had built and studied things on their own, and many of them had done research. I left that convention with one goal: find a way to do real research before I graduated.

Computer science had not always felt this uncertain. My mother is a software engineer, and she has been my biggest role model; watching her work made computing feel like something I could actually do. In middle school I joined a robotics club, and our team finished in the top 15 at the 2019 VEX Robotics World Championship. That was the first time I felt the satisfaction of solving a hard problem under pressure. My first computer science classes at UCF were a different kind of pressure. The projects were a bit of a trial by fire, pushing concepts my mind was not used to thinking about yet, but they taught me how to stay calm when things were not going the way I expected and to think a problem through instead of panicking. Outside of class, I found that I especially loved building for mobile devices. At a hackathon, my team built Car-Bon, a phone app that reads live data from a car's OBD-II sensor and estimates the car's carbon emissions during a drive from its RPM, engine load, and speed. Projects like that were fun, but by Fall 2024 I knew they were not enough to show anyone what I could really do.

Intellectual Merit

Finding research. My first real chance came through a paid, NSF-funded REU at Florida International University in Summer 2025, and it changed how I saw computer science as a whole. Our team studied whether synthetic speech could make up for how little clinical audio data exists for depression detection. I helped build the pipeline that turned real transcripts into synthetic speech with several text-to-speech systems (Coqui, Gemini, and Chatterbox), including a path for the Mandarin EATD-Corpus, and then trained spectrogram-based classifiers on mixes of real and synthetic data. A 75% synthetic-to-real mix worked best (79.8% accuracy, F1 0.85), while training on only synthetic data made things worse. I am a co-author on the resulting paper, *LLM-Generated Synthetic Samples on Speech Analysis for Health*, now in preparation for IEEE Signal Processing Letters. Reading papers, building a system, and writing up what we found hooked me in a way that no class project ever had.

Turning to security. That fall, my senior design capstone with the Georgia Tech Research Institute (GTRI) pulled me toward security. Our seven-person team built DocTalk, an AI healthcare assistant where an "orchestrator" agent hands tasks to specialized sub-agents, and then we spent the year attacking and defending it. I am a shy, introverted person, and group work used to be my least favorite part of computer science; on a team where six other people depended on my infrastructure every day, I learned to communicate early and often, and these days I actually enjoy it. As lead infrastructure developer, I set up the containerized environment that made our experiments repeatable: Docker services behind an nginx reverse proxy, an isolated Kali Linux attacker container, mock outside services, and CI automation for the team. I was also one of two engineers responsible for the remote code execution attack. A few innocent-looking questions got the orchestrator to reveal that its statistics agent ran any Python code it was given, as root, which meant an attacker could rewrite the system's own agent instructions. I also worked on the defensive side, and figuring out how to stop attacks my own teammates had designed turned out to be the part of the project I enjoyed most. That remote code execution attack worked 83% of the time against the undefended system, 5% after we hardened the prompts, and 0% after we added layered technical defenses. Watching our defenses actually shut that attack down was exciting, but the 5% stuck with me more: a system can look safe at the prompt level and still fail underneath. We released DocTalk, its full attack suite, and recorded walkthroughs of each attack as open source so others could test their own ideas against it. Around the same time, in a graduate course, I built a repeatable threat-modeling workflow combining data flow diagrams with STRIDE and a simple risk score. On a cloud-backed web app, it found 23 threats compared with 9 from an informal review and cut triage time by 40%, flagging problems like exposed API keys, incomplete login token validation, overly permissive database rules, and personal information leaking into logs. That project taught me that a repeatable, measurable process beats a smart guess.

Joining the Sage Research Lab. In my final year, I took a graduate software engineering course with Dr. Kevin Moran, whose work on AI-assisted software engineering and mobile apps had already caught my interest. When he assigned a research paper to analyze and present, I treated it as the most important assignment I had ever done. Afterward, he told me my analysis was on par with that of current professors and asked if I had considered a PhD. This was the moment I had been waiting for since SHPE, and I did not hesitate. I joined his lab and was admitted to UCF's doctoral program with the ORCGS Doctoral Fellowship. My first work there supported a study, led by senior PhD students and since submitted to ICSE, on how the quality of a bug report affects whether AI can repair the bug. I wrote structured bug reports for the dataset (the visual defect or crash, the screen it appears on, and the exact steps to reproduce it), helped develop and apply a taxonomy for grading AI repairs, and validated the scripts that reproduced each bug on Android emulators. Across 50 bugs in 50 open-source apps, only 25 AI repairs were truly correct. Ten fixed the symptom but not the cause, and ten made the bug *look* fixed while it was still there. A pass/fail check would have missed most of that. Sitting in regular meetings with the senior students and outside collaborators, I got to see how a research project really runs: how priorities shift when results come in, how much depends on small details like a precise reproduction step, and how much is at stake when a paper goes out. It also showed me the problem I want to spend my PhD on.

My research direction. That study is where my proposed research comes from: if AI agents cannot tell whether their own repairs are correct, neither can the developers who trust them. I call this the oracle problem for UI repair. My plan is a verification oracle that reads what a bug report expects, finds the specific UI component it describes, and checks whether that component changed the way the report asked for, instead of accepting any visible change as a fix. Two other projects follow the same thread. In my Advanced Software Systems Security course, I lead a four-person team studying build-pipeline attacks like SolarWinds and the XZ Utils backdoor; I built our engine, including a Meson build-file parser, a library of mutation operators, and a Docker build harness that runs each of 500+ planned attack mutants and classifies its effect, so we can measure whether an existing defense blocks, detects, or misses it. In the lab, I contribute to SpecMine, a dataset of spec-driven development artifacts mined from GitHub for the MSR Mining Challenge. All three ask the same question: where do automated systems fail quietly, producing results that look right while hiding real errors?

Why I am prepared for this work. The oracle problem sits where my experiences meet. DocTalk and my threat-modeling work taught me to think like an attacker and to never trust a system just because its output looks right. The repair study gave me hands-on experience with exactly the kind of Android bugs, bug reports, and emulator-based reproduction my research builds on. And my mobile projects mean I already know how these apps are put together. The Sage Research Lab is the right place to do this: our group developed FRAME, the UI-understanding model my oracle builds on, and the structured bug reports and reproduction scripts from the repair study give me a head start on the benchmark. With Dr. Moran's guidance and the lab's infrastructure, I have what I need to carry out the plan in my Graduate Research Plan.

Broader Impacts

AI coding agents are quickly becoming part of how software gets built, often with little human review. An agent that confidently reports a bug as fixed when it is not is a real risk anywhere reliability matters, from medical software to financial systems. Because AI and cybersecurity are national priorities, these failures are also a matter of national security and economic competitiveness, and my work with GTRI showed me how much research like this benefits from collaboration between universities, industry, and government labs. I care most about making those risks visible and fixable. I believe the best defense is one users barely notice: people should not have to change how they work, or become security experts, to stay safe. That means building checks directly into the tools developers already use. I have already started doing this kind of work in the open: DocTalk and its attack suite are public, as is my threat-modeling workflow and its artifacts. During my PhD, I will release the BugFix-Oracle benchmark and oracle as open-source tools that teams can add to their CI pipelines, report systematic failures I find to the developers of the AI tools involved, and share the results through developer-facing talks and plain-language writing, so the people adopting AI agents know when not to trust them.

My own path also shows what public investment in research can do. At SHPE, I saw how many Hispanic students are entering computing and how few have had a research experience. Mine came through an NSF-funded REU, which gave me mentorship I could not have found on my own. Building a career as a Hispanic researcher in industrial AI security is one way I can show other students that this path is realistic. As a Graduate Teaching Associate for CEN 5016, UCF's graduate software engineering course, I grade assignments and answer students' questions. That work keeps me grounded in how developers actually use, and trust, the tools my research studies.

Future Goals

After my PhD, I want to join a research and development group at a major technology company and push what is possible in AI security: finding the places where AI systems and AI-written software fail in ways that look like success, and building defenses that hold up at scale. Industry R&D is where those defenses reach the most people the fastest. One problem I already want to chase is the security of spec-driven development, which I started thinking about while working on SpecMine. In this style of development, a single specification file tells AI agents how an entire system should be built and maintained. If an attacker can quietly edit that file, every agent that reads it could be steered into building something harmful, and today there is little stopping one compromised spec file from affecting the others. Problems like this, where the attack surface is the AI workflow itself, are exactly what I want to work on. Along the way, I want to keep helping developers and organizations understand the real dangers of AI vulnerabilities. Two years ago, I left a convention afraid that I had nothing to show anyone. Since then, I have helped write my first research paper, attacked and defended a real AI system, and found a research problem I care about. The NSF Graduate Research Fellowship would give me the freedom to focus fully on that problem during my PhD and build the foundation for the career I want.